AUDIT :
Erreur sur le Cube OLAP

SUPER SMART MARKET

Présentation de la mission

Le constat: CA Instable

Ecart de lecture du CA de 9 057€ pour le 14 août 2024

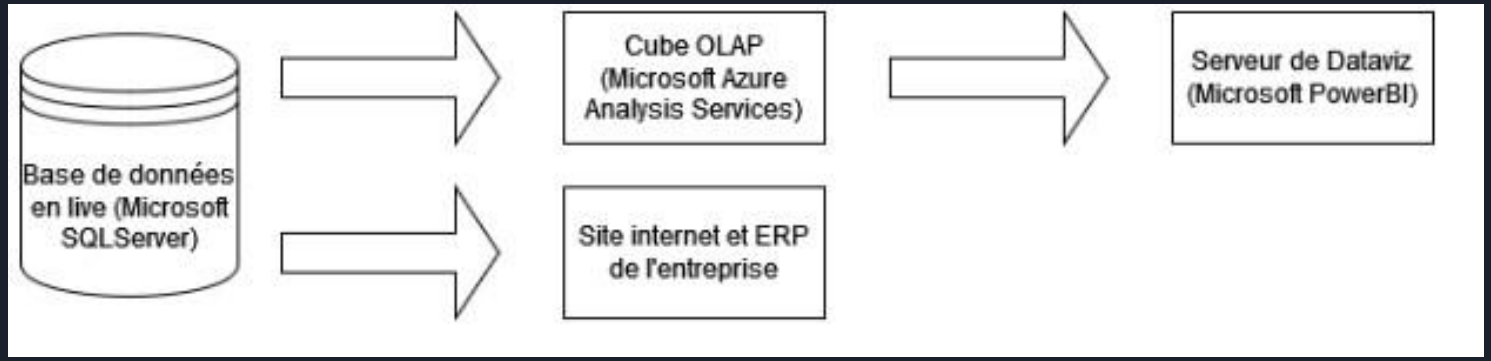
Hypothèse: L’erreur provient du Cube OLAP

Mission: Auditer et Corriger

1. Confirmer l’anomalie
2. Identifier les failles
3. Proposer une solution

Analyse de la base de données

Architecture de l'entreprise

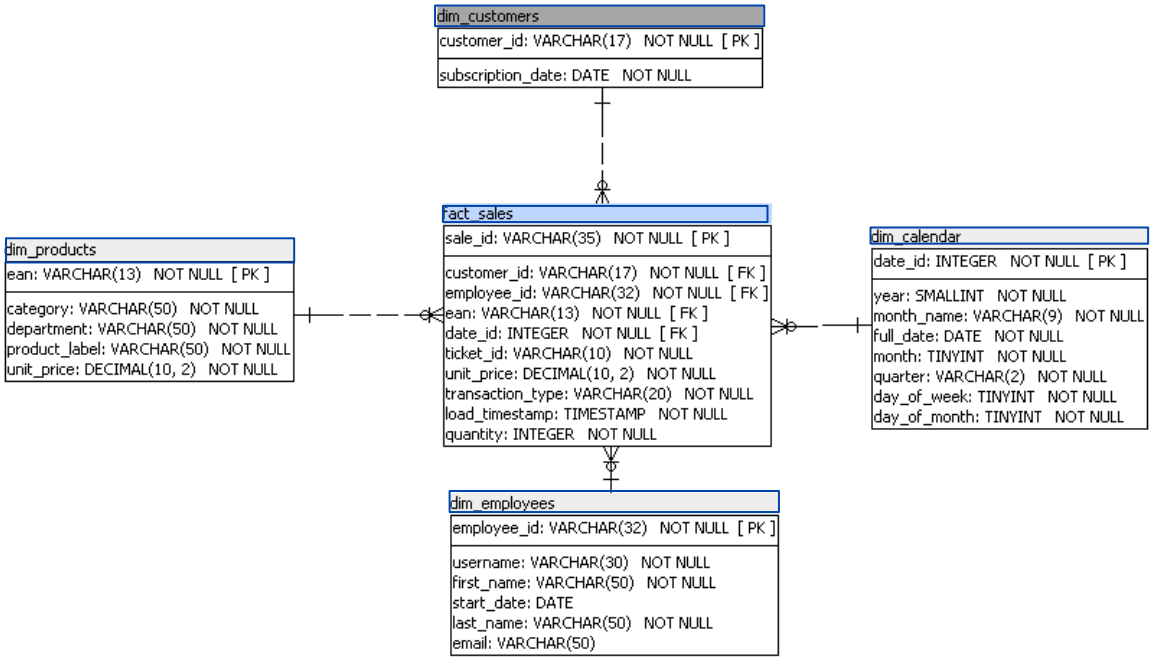


Le Parcours de la Donnée

1. Source Opérationnelle (OLTP)
2. Entrepôt de Données (OLAP)
3. Outil de Visualisation (BI)

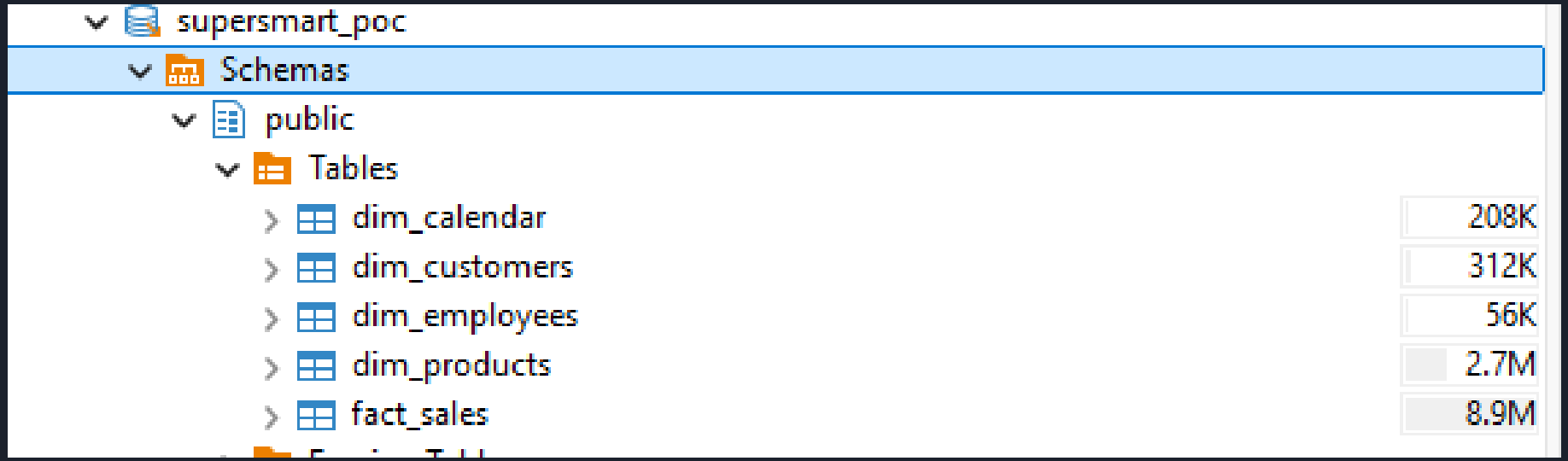
Analyse de la BDD

Le Modèle en Etoile Proposé



Présentation du prototype de BDD

Explication de la base de données



Résultat des requêtes SQL

Le chiffre d'affaires total pour le 14 Août

```sql
/* ⏮ V1 — AVANT */
SELECT SUM(p.unit_price)
FROM fact_sales f
JOIN dim_calendar c ON c.date_id = f.date_id
JOIN dim_products p on p.ean = f.ean
WHERE c.day_of_month = 14 AND c.month = 8 AND c.year = 2024;
-- Résultat : 284 243.88


/* ⏭ V2 — APRÈS */
SELECT SUM(f.unit_price * f.quantity) AS total_sales
FROM fact_sales f
JOIN dim_calendar c ON c.date_id = f.date_id
WHERE c.full_date = '2024-08-14';
-- Résultat : 284 243.88
```

Résultat des requêtes SQL

Le chiffre d'affaires par client pour le top 10 des clients

```sql
-- Calculer le CA par client pour le top 10 clients
SELECT f.customer_id, SUM(p.unit_price) as ca_par_client
FROM public.fact_sales f
JOIN public.dim_products p ON f.ean = p.ean
GROUP BY f.customer_id
ORDER BY ca_par_client DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	customer_id character varying (17)	ca_par_client numeric
1	CUST-JNSOZSFORR88	846.86
2	CUST-GM6VBAYAB8SF	666.86
3	CUST-L2ST2JHI7K9O	644.18
4	CUST-WU7ZKQJE4L17	608.93
5	CUST-9WM83101QDTI	582.03
6	CUST-ZMAOVX8XYGJY	576.39
7	CUST-3K66CV0OHH7Q	571.44
8	CUST-CG23SXJDRNYR	531.09
9	CUST-D8IOFHVUFX3Y	477.35
10	CUST-IHN1HQRI7PYJ	463.73

Résultat des requêtes SQL

La part de chiffre d'affaires encaissé par employé

```sql
-- Calculer la part de CA encaissé par employé.
WITH ca_employes AS(
    SELECT e.first_name, e.last_name, SUM(p.unit_price) as ca_par_employe
    FROM fact_sales f
    JOIN dim_products p ON p.ean = f.ean
    JOIN dim_employees e ON e.employee_id = f.employee_id
    GROUP BY e.employee_id, e.first_name, e.last_name
)
SELECT first_name, last_name,
        ((ca_par_employe / SUM(ca_par_employe) OVER()) * 100.00)::DECIMAL(5,2) AS pct_ca_employe
FROM ca_employes
ORDER BY pct_ca_employe DESC;
```

Data Output Messages Notifications

Showing rows: 1

	first_name character varying (50)	last_name character varying (50)	pct_ca_employe numeric (5,2)
1	Adélie	Boulet	2.75
2	Eugène	Jacquier	2.72
3	Charlène	Delisle	2.46
4	Pierre	Manoury	2.33
5	Tristan	Arsenault	2.28
6	Auriane	Dufresne	2.24
7	Abelin	Dutertre	2.16
8	Pierre	Ange	2.15
9	Arnaud	Lièvremont	2.15
10	Patricia	Rodier	2.14
11	Solène	Deslys	2.11

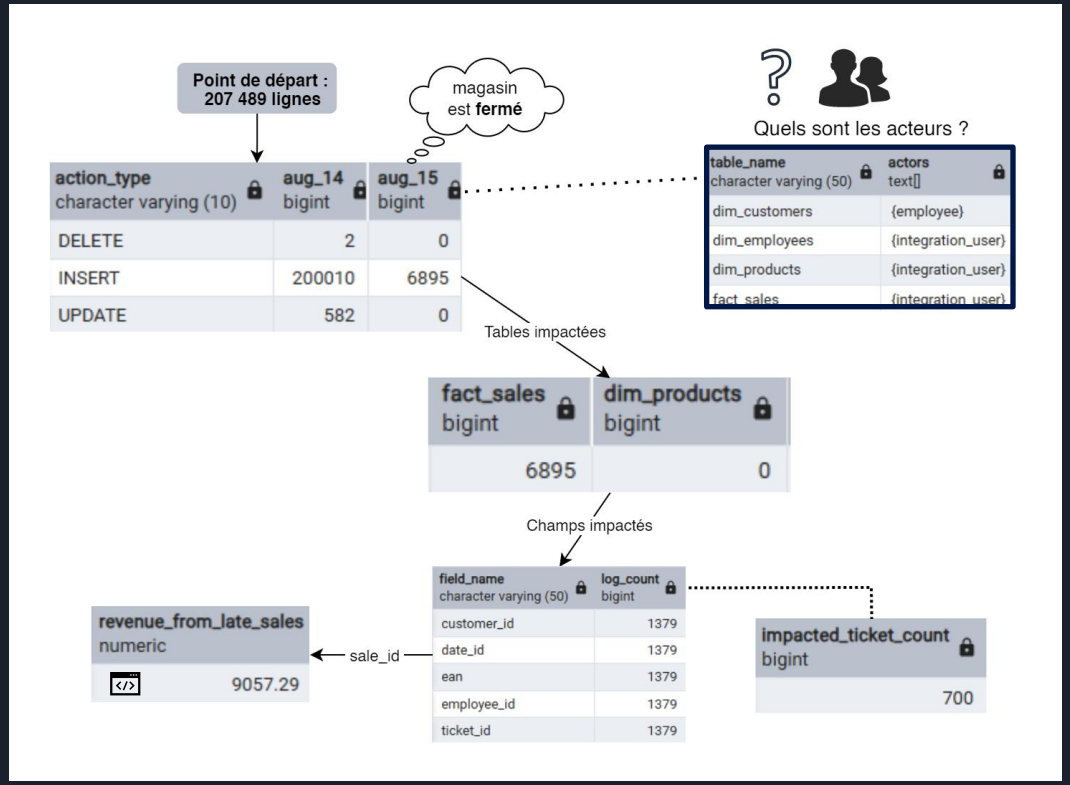
Analyse des logs
Dictionnaire des données et explications

LOGS (207 489, 8)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION	REGLE DE CALCUL
	log_id	Identifiant unique du log.Auto incrémenté	SERIAL		PK	AUTO INCREMENT
id_user	user_id	Identifiant de l'utilisateur ayant fait l'action.	VARCHAR(32)			
date	log_date	Date de l'action.	DATE			
action	action_type	Type d'action (INSERT, UPDATE, DELETE).	VARCHAR(10)			
table_insert	table_name	Table impactée (dim_products, fact_sales...).	VARCHAR(50)			
id_ligne	row_id	ID de la ligne impactée (EAN, sale_id...).	VARCHAR(50)			
champs	field_name	Champ modifié (ex: unit_price).	VARCHAR(50)			
detail	new_value	La nouvelle valeur, stockée en texte pour flexibilité	VARCHAR(255)			

Analyse des logs

La Piste des ventes tardives



Recommandations pour l’entreprise

Figer et enrichir la table de faits pour garantir la stabilité du CA

Actions proposées : ajouter des champs complémentaires dans la table de fait

- quantity
- unit_price

Table fact_sales

- transaction_type
- load_timestamp

Démonstration :
Impact d'une mise à jour de prix sur l'ancienne et la nouvelle méthode de calcul.

Démonstration :
Comment le nouveau modèle permet d'isoler et de quantifier les ventes tardives.

Recommandations pour l’entreprise

Avantages et inconvénients de la solution

Avantages :

- Fiabilité Absolue du CA : Le Chiffre d'Affaires devient stable et historiquement juste.
- Performance améliorée : Le calcul du CA se fait sans jointure
- Transparence : Les ventes tardives sont maintenant identifiables et quantifiable

Inconvénients :

- Modification de l’ETL : L'ETL doit être adapté pour gérer la récupération du prix, le calcul de la quantité, la classification de la transaction, etc.
- Augmentation du Stockage

Recommandations pour l’entreprise

Sécurisation et simplification du modèle

Actions proposées : Exclusion des champs sans valeurs analytiques

Table dim_employees

- hash_mdp
- email ?

full_date

Table dim_calendar

annee_mois

Démonstration

Recommandations pour l’entreprise

Avantages et inconvénients de la solution

Avantages :

- Sécurité et conformité au RGPD
- Modèle simplifié et plus lisible

Inconvénients :

- Impact sur l’existant (annee_moi et email)
- Effort de mise en ouvre : Modification de l’ETL
- Validation métier requise pour la colonne email.

Recommandations pour l’entreprise

Automatiser la maintenance de la table dim_calendar

Action proposée : Création d’une procédure stockée pour la mise à jour autonomes de la dimension calendrier.

sp_populate_calendar

Table dim_calendar

Démonstration

Recommandations pour l’entreprise

Avantages et inconvénients de la solution

Avantages :

- Autonomie et agilité accrues: L’ équipe Data devient autonome pour étendre la place d’analyse.
- La génération de données coté serveurs de BDD.

Inconvénients :

- Dépendance à la technologie SGBD.

Recommandations pour l’entreprise

Mettre en place une stratégie d’indexation évolitive

Action proposée :
Définir et appliquer une stratégie d’indexation selon le besoin métier.

Indexation sur les clés étrangères
et sur le champ full_date

BDD OLAP

Démonstration

Recommandations pour l’entreprise

Avantages et inconvénients de la solution

Avantages :

- Performance drastiquement améliorée
- Expérience utilisateur optimale pour les analystes

Inconvénients :

- Consommation d’espace de stockage
- Impact sur le temp de chargement

Recommandations pour l’entreprise

Implications pour l’organisation actuelle

Equipe BI :

- Confiance Restaurée : CA stable
- Productivité Accrue : Moins de temps passé à investiguer des anomalies
- Nouvelle Possibilités Analytiques : Taille du panier, analyse des ajustements

Equipe Data :

- Maintenance Simplifié : Un système robuste est plus facile à maintenir et faire évoluer

Recommandations pour l’entreprise

Planning de mise en place et coût éventuel

Phase 1 : Validation et Conception (Durée estimé : 1-2 semaines)

- Présenter l’audit aux équipes (BI, Data) pour valider les constats et s’aligner sur les actions.
- Rédiger les spécifications techniques détallées de la nouvelle architecture et du nouvel ETL.

Phase 2 : Développement de Tests (Durée estimé : 4-6 semaines)

- Déploiement sur un environnent de test
- Planifier et tester la migration des données historiques

Phase 3 : Déploiement et Formation (Durée estimé : 1 semaine)

Coût éventuel :
Le coût principal sera l'investissement en temps humain,
l'impact sur l'infrastructure existante étant considéré comme faible.

Conclusion

```sql
/* ⏮ V1 — AVANT */
SELECT SUM(p.unit_price)
FROM fact_sales f
JOIN dim_calendar c ON c.date_id = f.date_id
JOIN dim_products p on p.ean = f.ean
WHERE c.day_of_month = 14 AND c.month = 8 AND c.year = 2024;
-- Résultat : 284 243.88


/* ⏭ V2 — APRÈS */
SELECT SUM(f.unit_price * f.quantity) AS total_sales
FROM fact_sales f
JOIN dim_calendar c ON c.date_id = f.date_id
WHERE c.full_date = '2024-08-14';
-- Résultat : 284 243.88
```

Analyse de la BDD

Le Dictionnaire des données (Produits | dim-products)

PRODUITS (18 040, 5)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION
EAN	ean	Numéro EAN du produit.	VARCHAR	13	PK, NOT NULL
categorie	category	Catégorie générale du produit (ex: Produits Secs, Hygiène).	VARCHAR	50	NOT NULL
Rayon	department	Rayon spécifique où se trouve le produit.	VARCHAR	50	NOT NULL
Libelle_produit	product_label	Nom commercial du produit.	VARCHAR	50	NOT NULL
prix	unit_price	Prix de vente unitaire du produit.	DECIMAL(10,2)		NOT NULL, CHECK (unit_price >= 0)

Analyse de la BDD

Le Dictionnaire des données (Clients | dim-customers)

👥 CLIENTS (2 297, 2)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION
CUSTUMER_ID	customer_id	Identifiant unique du client.	VARCHAR	17	PK
date_inscription	subscription_date	Date à laquelle le client s'est inscrit au programme de fidélité ou à la base de données.	DATE		NOT NULL, Format:AAAA-MM-JJ

Analyse de la BDD

Le Dictionnaire des données (Table de fait: Ventes)

VENTES (41 377, 6) (TABLE DE FAIT)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION
ID_BDD	sale_id	Identifiant unique de chaque ligne (enregistrement de vente).	VARCHAR	35	PK, NOT NULL
CUSTOMER_ID	customer_id	Identifiant unique du client ayant effectué l'achat.	VARCHAR	17	FK vers clients, NOT NULL
id_employe	employee_id	Identifiant de l'employé qui a encaissé la vente.	VARCHAR	32	FK vers employé
EAN	ean	Numéro EAN (European Article Number) du produit vendu.	VARCHAR	13	FK vers produits, NOT NULL
Date achat	date_id	Date de l'achat stockée au format numérique.	INTEGER		FK vers calendrier
ID ticket	ticket_id	ID du ticket de caisse. (Un ticket peut contenir plusieurs produits)	VARCHAR	10	Permet d'agréger les ventes par transaction.
	unit_price	Prix de vente du produit au moment de la transaction	DECIMAL(10,2)		NOT NULL, CHECK (unit_price >= 0)
	transaction_type	Identifie le type de la transaction.(Initiale ou ajustement)	VARCHAR	20	CHECK (transaction_type IN ('INITIAL_SALE', 'ADJUSTMENT'))
	load_timestamp	Assure la traçabilité du chargement	TIMESTAMPTZ		
	quantity	Quantité de l'article vendu pour cette ligne de transaction.	INTEGER		CHECK (quantity > 0)

Analyse de la BDD

Le Dictionnaire des données (Employé | dim-employees)

EMPLOYE (56, 7)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION
id_employe	employee_id	Identifiant unique de l'employé. HASH MD5	VARCHAR	32	PK, NOT NULL
employe	username	Nom d'utilisateur/login de l'employé.	VARCHAR	30	NOT NULL, UNIQUE
prenom	first_name	Prénom de l'employé.	VARCHAR	50	NOT NULL
nom	last_name	Nom de famille de l'employé.	VARCHAR	50	NOT NULL
date_debut	start_date	Date d'embauche stockée au format numérique dans le fichier source	DATE		
hash_mdp	✕	Hash du mot de passe de l'employé. (MD5)	VARCHAR	32	A SUPPRIMER (DONNES SENSIBLE ET INUTILE DANS LA CUBE OLAP)
mail	email	Adresse email professionnelle de l'employé. (Vraiement besoin?)	VARCHAR	50	

Analyse de la BDD

Le Dictionnaire des données (Calendar | dim-calendar)

CALENDAR (1 999, 8)

CODE(SOURCE)	CODE (FINAL)	SIGNIFICATION	TYPE	LONGUEU	REGLE DE GESTION
date	date_id	Date au format numérique (typique des systèmes Microsoft Excel/SQL Server).	INTEGER		PK (Jointure avec date_id dans la table de fait)
annee	year	Année de la date.	SMALLINT		NOT NULL
mois	month	Numéro du mois (1 à 12).	TINYINT		CHECK (mois BETWEEN 1 AND 12)
Jour	day_of_month	Jour du mois [1-31]. ⚠ **Corrigé à partir de la colonne 'Jour' mal formatée**	TINYINT		CHECK (Jour BETWEEN 1 AND 31)
mois_nom	month_name	Nom du mois (ex: "janvier"). 12 valeurs uniques.	VARCHAR	9	
annee_mois	✕	Identifiant combinant année et mois (utilité pas claire!!!).	INTEGER		Le format actuel est peu clair.Exclu du POC
jour_semaine	day_of_week	Jour de la semaine (1 pour dimanche, 2 pour lundi... ou autre selon la norme utilisée).	TINYINT		NOT NULL, CHECK (jour_semaine BETWEEN 1 AND 7)
trimestre	quarter	Trimestre de l'année (Q1, Q2, Q3, Q4).	VARCHAR	2	NOT NULL, CHECK (trimestre IN ('Q1','Q2','Q3','Q4'))
	full_date	La version en type "date" du champ date_id, pour faciliter les requêtes	DATE		NOT NULL

Annexe 6

Requête finale pour mesurer la variation du CA.

```sql
SELECT
    SUM(p.unit_price) AS revenue_from_late_sales
FROM
    fact_sales s
JOIN
    dim_products p ON s.ean = p.ean
WHERE
    s.sale_id IN (
        -- Récupère la liste unique des sale_id
        -- qui ont été insérés dans les logs le 15 août.
        SELECT DISTINCT row_id
        FROM app_logs
        WHERE
            log_date = '2024-08-15'
            AND action_type = 'INSERT'
            AND table_name = 'fact_sales'
    );
```

Output Messages Notifications

revenue_from_late_sales numeric
9057.29

Annexe 7

Requête SQL de synthèse des acteurs

```sql
WITH per_action AS (
  SELECT
    action_type,
    BOOL_OR(user_id = 'integration_user_id') AS has_integration,
    COUNT(DISTINCT CASE WHEN user_id <> 'integration_user_id' THEN user_id END) AS employee_count
  FROM app_logs
  GROUP BY action_type
)
SELECT
  action_type,
  CASE
    WHEN has_integration AND employee_count > 0
        THEN 'integration_user_id + (' || employee_count || ' employees)'
    WHEN has_integration
        THEN 'integration_user_id'
    WHEN employee_count > 0
        THEN '(' || employee_count || ' employees)'
    ELSE 'Empty user_id'
  END AS users
FROM per_action
ORDER BY action_type;
```

Output Messages Notifications

action_type character varying (10)	users text
DELETE	integration_user_id
INSERT	integration_user_id + (18 employees)
UPDATE	integration_user_id

Annexe 8

Requête SQL des opérations détailles des acteurs

```sql
SELECT
        user_id,table_name ,
        ARRAY_AGG(DISTINCT field_name) AS impacted_fields,
        ARRAY_AGG(DISTINCT action_type) AS action_types,
        COUNT(log_id) AS log_count
FROM app_logs
-- WHERE user_id != 'integration_user_id'
GROUP BY user_id, table_name
ORDER BY log_count DESC
```

Output Messages Notifications

Showin

user_id character varying (32)	table_name character varying (50)	impacted_fields character varying[]	action_types character varying[]	log_count bigint
integration_user_id	fact_sales	{customer_id,date_id,ean,employee_id,ticket_id}	{INSERT}	206885
integration_user_id	dim_products	{unit_price}	{UPDATE}	575
integration_user_id	dim_employees	{hash_mdp,NULL}	{DELETE,UPDATE}	9
6fa61d0ecae0b563fef18d36b2039c8e	dim_customers	{subscription_date}	{INSERT}	2
dd595f0f0b3400df2908f0be7723dad4	dim_customers	{subscription_date}	{INSERT}	2
37c6a856b2e14217855d808fbc8e56a7	dim_customers	{subscription_date}	{INSERT}	1
43710c31fab3c974c906d48477bd4a83	dim_customers	{subscription_date}	{INSERT}	1

Annexe 9

Démonstration d'instabilité de CA suite à une mise à jour des prix catalogue

step_description text
ACTION: Tous les prix dans dim_products ont été augmentés de 15%.

row returned

Revenue (Legacy) numeric	Delta vs Truth (Legacy) numeric	Revenue (New) numeric	Delta vs Truth (New) numeric
326907.56	42663.68	284243.88	0.00

Annexe 10

Démonstration d'utilité de la colonne de commodité full_date

```sql
/* ⏮ V1 – AVANT */
WHERE c.day_of_month = 14 AND c.month = 8 AND c.year = 2024;
/* ⏭ V2 – APRÈS */
WHERE c.full_date = '2024-08-14';
```

Annexe 11

Démonstration de la traçabilité du CA après l'amélioration

```sql
SELECT
    SUM(f.quantity * f.unit_price) FILTER ( WHERE transaction_type = 'INITIAL_SALE') AS initial_sales,
    SUM(f.quantity * f.unit_price) FILTER ( WHERE transaction_type = 'ADJUSTMENT')   AS adjustments,
    SUM(f.quantity * f.unit_price) AS total_revenue
FROM fact_sales f
JOIN dim_calendar c ON c.date_id = f.date_id
WHERE c.full_date = '2024-08-14';
```

Output Messages Notifications

initial_sales numeric	adjustments numeric	total_revenue numeric
275186.59	9057.29	284243.88

Annexe 12

Exemple d’utilisation de la procédure stockée, sp_populate_calendar

```sql
BEGIN;

-- 1. On vérifie l'état initial : pas de données pour 2019
SELECT 'État initial' AS step, COUNT(*) AS count_2019 FROM dim_calendar WHERE year = 2019;

-- 2. On appelle la procédure pour générer les dates de 2019
CALL public.sp_populate_calendar('2019-01-01', '2019-12-31');
SELECT 'Procédure exécutée' AS step;

-- 3. On vérifie que les 365 jours de 2019 ont bien été ajoutés
SELECT 'État après appel' AS step, COUNT(*) AS count_2019 FROM dim_calendar WHERE year = 2019;
-- On peut même regarder à quoi ressemblent les données
SELECT * FROM dim_calendar WHERE year = 2019 LIMIT 5;

-- 4. On annule tout !
ROLLBACK;
SELECT 'Transaction annulée, base restaurée' AS final_status;
```

Output Messages Notifications

date_id [PK] integer	year smallint	month smallint	day_of_month smallint	month_name character varying (9)	day_of_week smallint	quarter character varying (2)	full_date date
43466	2019	1	1	janvier	2	Q1	2019-01-01
43467	2019	1	2	janvier	3	Q1	2019-01-02
43468	2019	1	3	janvier	4	Q1	2019-01-03
43469	2019	1	4	janvier	5	Q1	2019-01-04
43470	2019	1	5	janvier	6	Q1	2019-01-05

Annexe 13

Le champ annee_mois dans la table dim_calendar

Le fichier source

	A	B	C	D	E	F	G	H	I
1	date	annee	mois	Jour	mois_nom	annee_mois	jour_semaine	trimestre	
2	43831	2020	1	01	janvier	01/01/2020	4	Q1	
3	43832	2020	1	02	janvier	01/01/2020	5	Q1	
4	43833	2020	1	03	janvier	01/01/2020	6	Q1	
5	43834	2020	1	04	janvier	01/01/2020	7	Q1	
6	43835	2020	1	05	janvier	01/01/2020	1	Q1	
7	43836	2020	1	06	janvier	01/01/2020	2	Q1	
8	43837	2020	1	07	janvier	01/01/2020	3	Q1	

Affichage

```sql
SELECT FORMAT(CAST('2025-08-15' AS date), 'yyyy-MM') AS annee_mois;
```

Results | Messages

	annee_mois
1	2025-08

Analyse

Query | Query History

```sql
SELECT
    SUM(f.unit_price * f.quantity) AS total_revenue
FROM
    fact_sales f
JOIN
    dim_calendar c ON f.date_id = c.date_id
WHERE
    c.year = 2024 AND c.month = 8; -- C'est clair, simple et direct.
```

Data Output | Messages | Notifications

	total_revenue numeric
1	284243.88

Annexe 14

Les index crées dans le prototype

```sql
-- ==============================================================================
-- SECTION 4: CRÉATION DES INDEX POUR L'OPTIMISATION DES REQUÊTES
-- Objectif: Accélérer les jointures et les filtres, qui sont au cœur de
--           toutes les requêtes analytiques sur un schéma en étoile.
-- ==============================================================================
-- Index sur les clés étrangères de la table de faits
CREATE INDEX idx_fact_sales_ean ON public.fact_sales (ean);
CREATE INDEX idx_fact_sales_customer_id ON public.fact_sales (customer_id);
CREATE INDEX idx_fact_sales_employee_id ON public.fact_sales (employee_id);
CREATE INDEX idx_fact_sales_date_id ON public.fact_sales (date_id);

-- Index sur une colonne de filtre fréquente dans une dimension
CREATE INDEX idx_dim_calendar_full_date ON public.dim_calendar (full_date);
```